OOP Theory + Code Practice Assignment

Scenario: College Management System.

- students
- Trainers
- Admin Staff

⇒

- name
- age
- email

Phase 1 - functions Based Approach

Task 1: write functions for:

create_student()
show_student()

create_trainer()
show_trainer()

create_admin()
show_admin()

Student

Store:
- name
- age
- email
- course

Trainer

Store:
- name
- age
- email
- subject

Admin

Store:
- name
- age
- email
- department

Code:

```
def create_student (name, age, email, course):
    return {"name": name, "age": age, "email": email,
"course": course}

def show_student(student):
    Print ("--- student Detail ---")
    print("Name:", student["name"])
    Print ("Age:", student["age"])
    Print("Email", student['email])
```

Scanned with OKEN Scanner

```
    print("Course :", student["course"])

def create_trainer(name, age, email, subject):
    return {"name": name, "age": age, "email": email,
"subject": subject}

def show_trainer(trainer):
    print("--- Trainer Details ---")
    print("Name :", trainer["name"])
    print("Age :", trainer["age"])
    print("Email :", trainer["email"])
    print("Subject :", trainer["subject"])

def create_admin(name, age, email, department):
    return {"name": name, "age": age, "email":
email, "department": department}

def show_admin(admin):
    print("--- Admin Details ---")
    print("Name:", admin["name"])
    print("Age:", admin["age"])
    print("Email :", admin["email"])
    print("Department:", admin["department"])

student = create_student("Bonu", 21, "bhuvaneshwari@
                         gmail.com",
"Data Science")
show_student(student)
```

Scanned with OKEN Scanner

Output:

--- Student Details ---

Name : Bonu

Age : 21

Email : bhuvaneshwari@gmail.com

Course : Data Science

Theory Questions

1. which code is repeated in all functions?

The code that sets up and displays name, age, and email is repeated across every single function (create_student, create_trainer, create_admin, and their matching show functions).

2. Are name, age, and email repeated everywhere?

Yes, Every function explicitly accepts and manages name, age, and email, leading to duplicates dictionary keys and repeated print statements.

3. what happens if the company changes email validation logic?

If a rule is added (e.g., verifying that @ exists in email), we must manually update that validation inside create_student(), create_trainer(), and create_admin(). missing even one function creates bugs and inconsistent data.

4. Is maintenance becoming difficult?

yes. As more roles are added (e.g., Librarian, Accountant). every new entity requires its own pair of create/show

functions, making the codebase bloated and hard to track.

5. Can functions manage very large systems properly?

No, Procedural code with separate functions and data structures becomes messy and unmaintainable as system complexity grows.

6. What are the disadvantages of using only functions?

- Code Redundancy: Identical properties are redefined repeatedly.
- No Encapsulation: Data and logic live separately, making state tracking fragile.
- High Maintenance: A small change requires editing multiple functions.
- Poor Scalability: Adding new roles multiplies the amount of boilerplate code.

PHASE 2 :- Converting Into OOP

Task 2 - create Parent Class

create class Person:	Person Properties • name • age • email	Person Method Create: display_info()

```
class Person:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

    def display_info(self):
        Print (f"Name : {self.name}")
        print(f"Age : {self.age}")
        print (f"Email: {self.email}")
```

Scanned with OKEN Scanner

P = Person ("Bhuvaneshwari", 21, "bhuvaneshwari@gmail.com")

P. display_info()

output:

Name: Bhuvaneshwari

Age : 21

Email: bhuvaneshwari@gmail.com

Theory Questions

1. why do we create a parent class here?

We create the Person parent class to centralize common attributes and behavior shared by every individual in the college (students, Trainers, Admin staff).

2. what common properties are moved into the Parent class?

The properties name, age, and email are moved into the parent class alongside the display_info() method.

3. How does OOP reduce repeated code?

By bundling common attributes into a single parent class, child classes can inherit them automatically instead of redefining them from scratch.

4. Difference between functions and classes?

	Functions	classes
Concept	- Blocks of code that execute actions.	- Blueprints that Combine both data (attributes) and behavior (methods).

Scanned with OKEN Scanner

Data handling	- Data is passed as external arguments.	- Data is encapsulated within the object using self.
Reusability	- Reused by calling the function	- Reused through inheritance and instantiation.

PHASE 3 - Inheritance & Special Properties

Task 3 - create child classes

Create child classes:

Student
Trainer
Admin

using inheritance

Task 4 - Add special properties

Student	Trainer	Admin
Add:	Add:	Add:
• course	• subject	• department
method:	method:	method:
show_course()	show_subject()	show_department()

```
class Person:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

    def display_info(self):
        Print(f"Name: {self.name}")
        Print(f"Name: {self.age}")
        Print(f"Email: {self.email}")
```

Scanned with OKEN Scanner

```
class Student(Person):
    def show_course(self):
        Print(f"Course: {self.course}")

class Trainer(Person):
    def show_subject(self):
        Print(f"Subject: {self.subject}")

Class Admin(Person):
    def show_department(self):
        print(f"Department: {self.department}")
```

Theory Questions

1. what is Inheritance?

Inheritance is an OOP mechanism where a child class acquires the attributes and methods of an existing parent class.

2. why is inheritance useful?

- Code Reusability: write code once in the parent class and use it across all child classes.
- Extensibility: Easily add specialized features to child classes without altering Parent logic.
- Maintainability: Modifying Parent logic updates all inheriting classes automatically.

3. what Properties are inherited from the Parent class?

All attributes defined in Person (name, age, email) are inherited.

Scanned with OKEN Scanner

4. which methods are inherited automatically?

All public and protected methods defined in Person (such as display_info()) are inherited automatically.

5. Explain real-world examples of inheritance.

- Banking: A general Account class is inherited by SavingsAccount and CurrentAccount.
- E-commerce: An User class is inherited by Customer and Vendor.

PHASE 4 – Using super()

Task 5

Inside child constructors, use:

super().__init__()

```
class Person:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

    def display_info(self):
        print(f"Name : {self.name}")
        print(f'Age: {self.age}")
        print(f"Email : {self.email}")
```

Scanned with OKEN Scanner

```
Class Student(Person):
    def __init__(self, name, age, email, course):
        super().__init__(name, age, email)
        self.course = course

    def show_course(self):
        Print(f"Course : {self.course}")

s = Student("Bhuvaneshwari", 21, "bhuvaneshwari@gmail.com",
                                "Data science")
s.display_info()
s.show-course()
```

Output:

Name : Bhuvaneshwari
Age : 21
Email : bhuvaneshwari@gmail.com
Course : Data science

Theory Questions

1) why are we using super()?

super() provides access to Parent class methods from within a child class, allowing us to initialize parent attributes cleanly without re-writing initialization logic.

2) what repeated code is removed using super()?

we eliminate manually assigning self.name = name, self.age = age and self.email = email inside every child class constructor.

Scanned with OKEN Scanner

3. what happens if we do not use super()?

If a child class defines its own --init-- without calling super(), the parent's --init-- is overridden completely. As a result, inherited attributes (name, age, email) will not be created, triggering an AttributeError.

4. Does the Parent constructor execute automatically?

No, In python, overriding --init-- in a child class prevent class constructor from running unless explicitly called via super() or the Parent class name..

5. Explain constructor overriding.

Constructor overriding occurs when a child class defines its own --init-- method, replacing the default initialization behavior inherited from the Parent class.

PHASE 5 - Using Parent Class Name

Task 6

Replace:

super().--init--()

with:

Person.__init__(self, name, age, email)

Scanned with OKEN Scanner

```
class Person:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

class Student(Person):
    def __init__(self, name, age, email, course):
        Person.__init__(self, name, age, email)
        self.course = course
```

Theory Questions

1. Does this method also work?

Yes, explicitly calling Person.__init__(self, ...) works as long as self is passed manually as the first argument.

2. Difference between super() and Person.__init__()

- super(): Uses Python's Method Resolution Order (MRO) dynamically. Does not require passing self explicitly.
- Person.__init__(): Statically hardcodes the Parent class name and requires passing self explicitly.

3. Which is better for large projects?

super() is much better because it decouples child classes from specific parent class names, making future refactoring far easier.

Scanned with OKEN Scanner

4. which supports multiple inheritance properly?

super() supports multiple inheritance properly by honoring Python's MRO algorithm and preventing redundant base class initializations.

5. why is super() recommended in modern Python?

It avoids hardcoded dependencies, works seamlessly with complex multiple inheritance hierarchies, and reduces maintenance friction.

PHASE 6 - Parent Method Access

Task 7:

Call Parent methods using child object.

Example:

```
student.display_info()
student.show_course()
```

```
class Person:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

    def display_info(self):
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")
        print(f"Email : {self.email}")

class Student(Person):
    def __init__(self, name, age, email, course):
        super().__init__(name, age, email)
```

Scanned with OKEN Scanner

```
        self.course = course

    def show_course(self):
        print(f"Course : {self.course}")

student = Student("Bhuvaneshwari", 21, "bhuvaneshwari@gmail.com",
                  "Data science")
student.display_info()
student.show_course()
```

Theory Questions

1. why can child objects access parent methods?

Child objects can access Parent methods because inheritance automatically links the child classes scope to the parent classes namespace via Python's lookup rules.

2. what is method Inheritance?

Method inheritance is the ability of a derived class to execute non-overridden methods defined in its base class directly from child object instance.

3. Explain the relationship between parent and child classes.

It is an "Is-A" relationship. for instance, a Student IS-A Person. The child class is a specialized extension of the broader Parent class.

Scanned with OKEN Scanner

PHASE 7 - Types of Inheritance

Task 8 - Single Inheritance

Create:

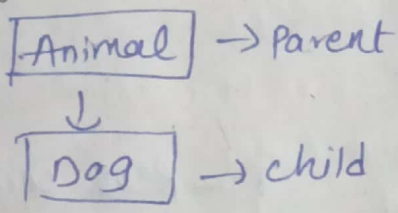
Animal → Dog

Dog should access Animal methods.

- Definition:

Single inheritance occurs when a derived class inherits directly from a single base class.

- Diagram:



- Code:

```
class Animal:
    def eat(self):
        print("Eating food...")

class Dog(Animal):
    def bark(self):
        print("Barking...")

d = Dog()
d.eat()
d.bark()
```

o/p: Eating food...
Barking...

- Real-world Examples

A basic Device parent class inherited by a Smartphone child class.

Scanned with OKEN Scanner

Task 9 - Multilevel Inheritance

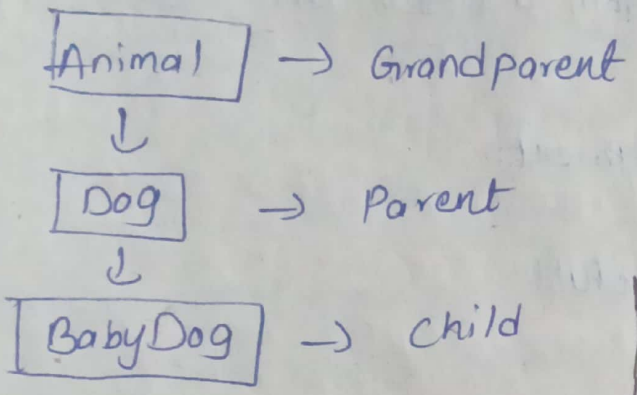
Create:

Animal → Dog → BabyDog

- Definition:

Multilevel inheritance occurs when a child class inherits from a parent class, which itself inherits from another base class, forming a chain.

- Diagram:



- Real-world Example:

Vechicle
↓
Car
↓
ElectricCar.

- Code:

```
class Animal:
    def eat(self):
        print("Eating...")

class Dog(Animal):
    def bark(self):
        print("Barking...")

class BabyDog(Dog):
    def weep(self):
        print("Weeping...")

bd = BabyDog()
bd.eat()
bd.bark()
bd.weep()
```

Scanned with OKEN Scanner

Task 10 - Hierarchical Inheritance

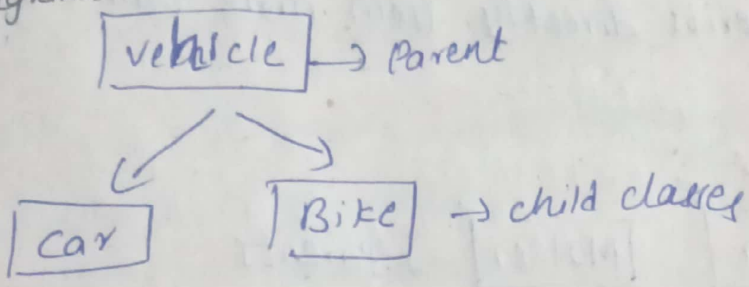
Create: Vehicle

Child classes:
- Car
- Bike

- Definition:

Hierarchical inheritance occurs when multiple child classes inherit directly from a single parent class.

- Diagram:



- Code:

```
class Vehicle:
    def start(self):
        Print("Vehicle starting...")

class Car(Vehicle):
    def drive(self):
        print("Driving car...")

class Bike(Vehicle):
    def ride(self):
        print("Riding bike...")

c = Car()
b = Bike()
c.start()
b.start()
```

- Real-World Example

A base User class inherited independently by Student, Trainer, and Admin.

Scanned with OKEN Scanner

Task 11 - Multiple Inheritance

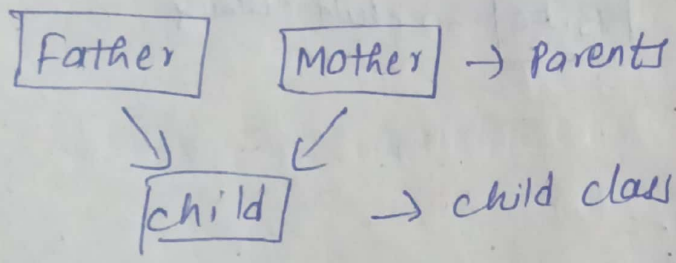
Create:
Father
Mother
Child

Child should inherit from both Parents.

- Definition:

Multiple inheritance occurs when a single child class inherits directly from more than one parent class.

- Diagram:



- Code:

```
class Father:
    def father_trait(self):
        print("Father's eyes")

class Mother:
    def mother_trait(self):
        Print("Mother's smile")

class Child(Father, Mother):
    Pass

C = Child()
c.father_trait()
c.mother_trait()
```

Scanned with OKEN Scanner

Advantages & Problems

- Advantages: Combines features from completely independent base classes into one class without code repetition.
- Problems (Diamond problem): Ambiguity can arise when multiple Parent classes share a method with the same name. Python resolves this using Method Resolution Order (MRO).

Task 12 – Hybrid Inheritance

Create a real-world example combining:

- multilevel Inheritance
- multiple Inheritance

- Definition:

Hybrid inheritance is a combination of two or more types of inheritance (e.g, mixing Multilevel and multiple inheritance).

- Why hybrid inheritance becomes complex

It becomes complex because managing multiple Parent paths can cause method lookup collisions and tricky initialization chains if not designed Carefully.

- Real-world use case

Combining a person → Employee (Multilevel) chain with a SportsPerson trait class to form an AthleteEmployee (Multiple).

Scanned with OKEN Scanner

- Code:

```
class Person:
    def identity(self):
        print("I am a Person")

class Employee(Person):
    def work(self):
        print("Working...")

class SportsPerson:
    def play(self):
        print("Playing sports...")

class CorporateAthlete(Employee, SportsPerson):
    pass

ca = CorporateAthlete()
ca.identity()
ca.work()
ca.play()
```

Final Reflection Questions

1. where did functions fail?

functions failed to encapsulate state (data) alongside behavior (methods). They caused code duplication across shared attributes (name, age, email) and created maintenance problems when business rules changed.

2. How did OOP solve the Problem?

OOP grouped shared attributes and behavior into a unified blueprint (Person class). Subclasses then

inherited common logic directly, eliminating boilerplate code.

3. why is inheritance powerful?

Inheritance enables code reusability, enforces logical hierarchical organization, and keeps applications extensible without breaking existing base logic.

4. why is code reuse important?

Code reuse speeds up development, minimizes bugs, ensures consistent logic across an application, and simplifies codebase updates.

5. why do we use constructors?

Constructors (__init__) initialize an object's state automatically when an instance of a class is created.

6. why do we use super()?

super() invokes base class constructors and methods dynamically without hardcoding class names, supporting proper method Resolution order (MRO).

7. Difference blw super() and ParentClass.__init__()

super() → resolves methods dynamically using MRO without requiring explicit self arguments.

ParentClass.__init__() → hardcodes the base class name and requires passing self manually.

8. which inheritance type was easiest?

Single Inheritance, because it flows in a straight line from one parent to one child.

9. which Inheritance type was difficult?

Hybrid/Multiple Inheritance, because tracking method lookup paths (MRO) and avoiding diamond-dependency conflicts requires extra caution.

10. How does OOP help in real-world Software Projects?

OOP modularizes large software platforms. Teams can work on Separate modules safely, easily extend existing features through inheritance and polymorphism, and maintain large codebases cleanly over time.

Scanned with OKEN Scanner